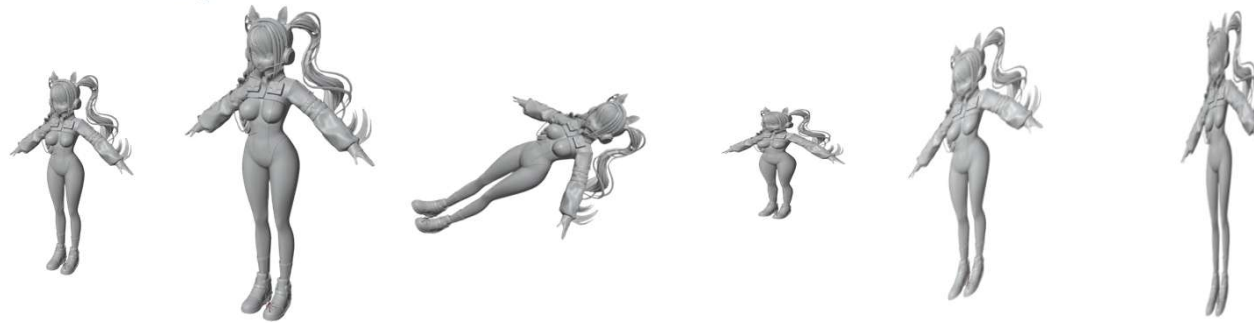


Directx3D11 프로그래밍

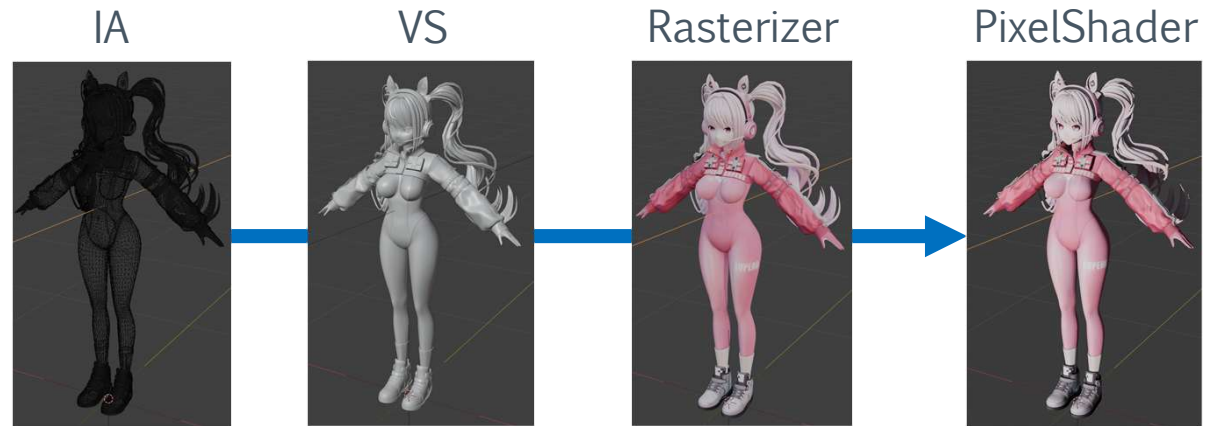
2. Transform

제작자 : 이창진
이메일 : ckdwls525@gmail.com



학습목표

```
// 공통 카메라(View/Proj)로 3개의 상수  
버퍼 엔트리를 준비합니다  
ConstantBuffer base;  
base.world = XMMatrixIdentity();  
base.view =  
XMMatrixTranspose(XMMatrixLookAtLH(  
XMVectorSet(m_CameraPos.x,  
m_CameraPos.y, m_CameraPos.z, 0.0f),  
XMVectorSet(m_CameraPos.x + 0.0f,  
m_CameraPos.y + 0.0f, m_CameraPos.z  
+ 1.0f, 0.0f),  
XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f)  
));  
float fovRad =  
XMConvertToRadians(m_CameraFovDeg);  
base.proj =  
XMMatrixTranspose(XMMatrixPerspectiv  
eFovLH(fovRad, AspectRatio(),  
m_CameraNear, m_CameraFar));
```



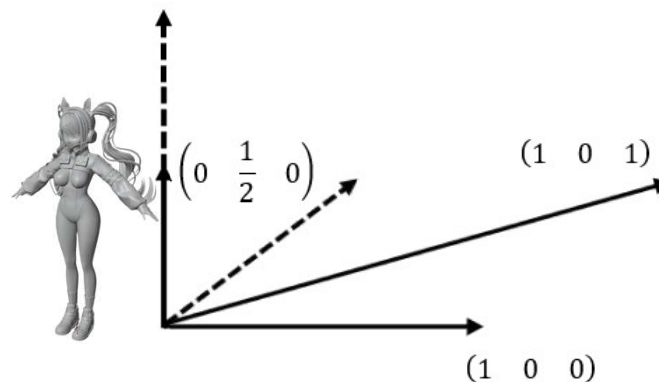
- › 1. DirectXMath 라이브러리의 행렬에 대해 이해합니다
- › 2. VertexShader의 World, View, Proj 변환에 대해 이해합니다
- › 3. Constant Buffer (상수버퍼)의 사용법을 학습합니다
- › 4. Rasterizer의 Viewprot를 이해합니다

DirectXMath Transform

Direct3D에서 행렬은 일반적으로 행 행렬로 표현됩니다

Directx의 경우에 3x3 행렬의 세 행 벡터는 위에서 아래로, 변환된 좌표 기저의 x'축(또는 벡터 $\vec{i}'$), y'축(또는 벡터 $\vec{j}'$), z'축(또는 벡터 $\vec{k}'$)입니다

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} \leftarrow \vec{i}' \rightarrow \\ \leftarrow \vec{j}' \rightarrow \\ \leftarrow \vec{k}' \rightarrow \end{bmatrix} = \begin{bmatrix} x' & y' & z' \end{bmatrix}$$



$$\begin{bmatrix} 2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{2} & 0 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \end{bmatrix}$$

DirectXMath Transform - Scale

스케일링은 객체의 크기를 변경하는 것을 의미합니다.

x축, y축, z축의 스케일을 변경하면 객체의 원하는 크기, 너비, 높이, 두께를 수정하거나 가져올 수 있습니다

DirectXMath에서 스케일링 행렬에 해당하는 함수는 다음과 같습니다. `XMMatrixScale`



$$S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & S_z \end{bmatrix}$$

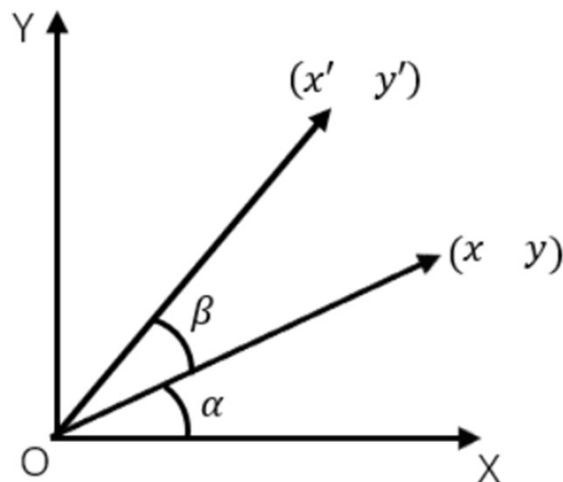


DirectXMath Transform - Rotation

회전은 객체의 방향을 변경합니다.

회전을 사용하는 방법을 배울 때 종종 x, y, z축을 중심으로 한 변환을 사용합니다.

DirectX Math에서는 회전 변환이 모두 **시계 방향으로** 이루어진다는 점에 유의해야 합니다



변환 전 벡터는 극좌표로 다음과 같이 표현됩니다.

$$x = r \cos \alpha$$

$$y = r \sin \alpha$$

β도 반시계 방향으로 회전한 후의 벡터는

극좌표로 다음과 같이 표현됩니다

$$x' = r \cos(\alpha + \beta) = r \cos \alpha \cos \beta - r \sin \alpha \sin \beta = x \cos \beta - y \sin \beta$$

$$y' = r \sin(\alpha + \beta) = r \sin \alpha \cos \beta + r \cos \alpha \sin \beta = x \sin \beta + y \cos \beta$$

위의 변환 과정은 다음과 같은 행렬로 표현될 수 있습니다

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} \cos \beta & \sin \beta \\ -\sin \beta & \cos \beta \end{bmatrix} = \begin{bmatrix} x' & y' \end{bmatrix}$$



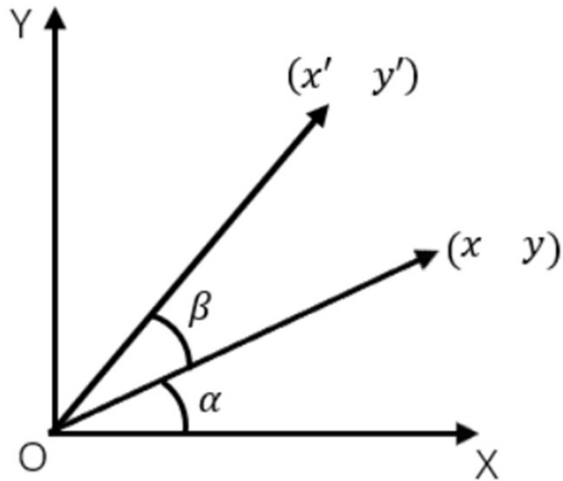
DirectX Math Transform - Rotation

DirectX Math에서는 회전 변환이 모두 **시계 방향으로** 이루어진다는 점에 유의해야 합니다

DirectX Math에서 스케일링 행렬에 해당하는 함수는

XMMatrixRotationX, **XMMatrixRotationY**, **XMMatrixRotationZ** (라디안) 입니다

여기서 우리는 3D 공간에서 X, Y, Z축을 중심으로
반시계 방향으로 회전하는 행렬 도출 과정을 생략 하고
다음의 세 가지 대응 행렬을 얻을 수 있습니다



$$\mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & \sin\theta \\ 0 & -\sin\theta & \cos\theta \end{bmatrix}, \mathbf{R}_y = \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix}, \mathbf{R}_z = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

왼손 좌표계는 오른손 좌표계의 **거울상이므로**

오른손 좌표계의 축을 중심으로 반시계 방향으로 회전하는데

사용되는 행렬 (축 방향의 원점을 바라볼 때)은 왼손 좌표계의

같은 축을 **중심으로 시계 방향으로 회전하는** 데 사용되는

행렬 (축 방향의 원점을 바라볼 때)과 동일하다는 점에 유의 **하세요**

$$\begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 1 & 0 \end{bmatrix} \begin{bmatrix} \cos 45^\circ & \sin 45^\circ & 0 & 0 \\ -\sin 45^\circ & \cos 45^\circ & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}$$

DirectXMath Transform - affine space

affine space는 4*4 행렬을 사용합니다.

왜 4*4 여야 할까요? w값이 필요한 이유는 무엇일까요?

선형 변환만으로는 필요한 모든 변환을 표현할 수 없으므로,
더 넓은 사상 범위를 가진 아핀 변환이라는 함수 클래스로 확장합니다.

두 좌표점의 차이를 **벡터(w 성분 0)**로 표현하고,
점과 벡터의 합을 **점(w 성분 1)**으로 표현하는 데 편리합니다

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} + \begin{bmatrix} t_x & t_y & t_z \end{bmatrix} = \begin{bmatrix} x' & y' & z' \end{bmatrix}$$



아핀 변환을 사용하면
위의 식을 아래의 식으로 변환할 수 있습니다.

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ a_{31} & a_{32} & a_{33} & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} = \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix}$$

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$$

이동행렬

$$\mathbf{T}^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -t_x & -t_y & -t_z & 1 \end{bmatrix}$$

이동행렬의 역행렬

DirectXMath World Transform

객체가 서로 다를 경우 다른 월드 변환 행렬을 사용해야 합니다.

동일한 객체의 모든 정점에는 변환을 위해 일관된 월드 변환 행렬을 사용해야 합니다

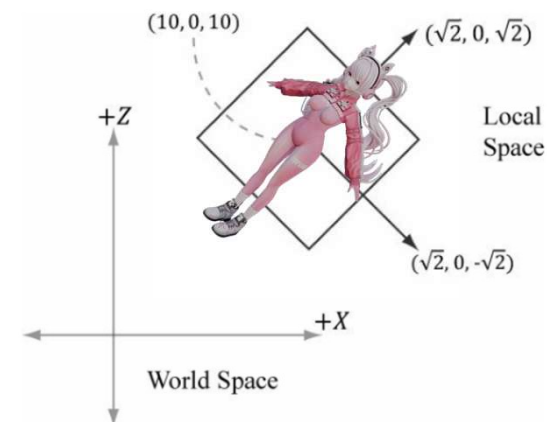
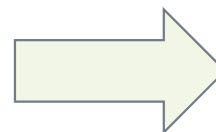
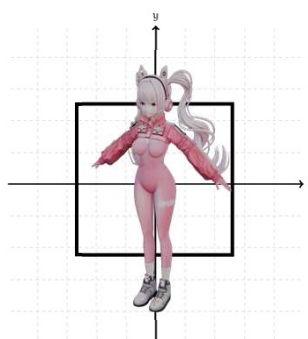
$[ux, uy, uz, 0], [vx, vy, vz, 0], [wx, wy, wz, 0]$
 이 세개는 로컬 좌표계의 서로 수직인 세 축 벡터를 나타냅니다
 (이 세 벡터는 반드시 크기가 같지는 않습니다)

$$S = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} \sqrt{2}/2 & 0 & -\sqrt{2}/2 & 0 \\ 0 & 1 & 0 & 0 \\ \sqrt{2}/2 & 0 & \sqrt{2}/2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

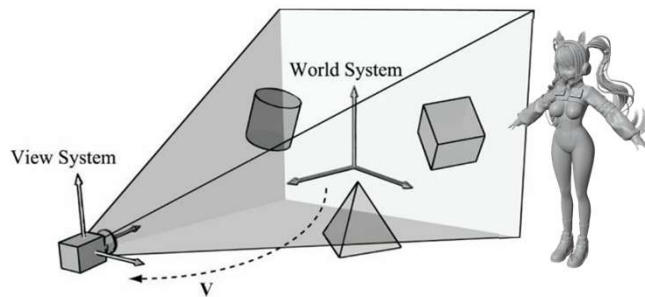
$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 10 & 0 & 10 & 1 \end{bmatrix}$$

$$W = SRT = \begin{bmatrix} \sqrt{2} & 0 & -\sqrt{2} & 0 \\ 0 & 1 & 0 & 0 \\ \sqrt{2} & 0 & \sqrt{2} & 0 \\ 10 & 0 & 10 & 1 \end{bmatrix}$$



DirectXMath View Transform

2D 이미지를 얻으려면 가상 카메라라는 개념을 도입해야 합니다.
가상 카메라는 관찰 좌표계이자 국소 좌표계로 볼 수 있습니다.
원점은 카메라의 위치,
Z축은 카메라의 관찰 방향, Y축은 카메라의 위쪽 방향, X축은 카메라의 오른쪽 방향입니다



World 좌표계에서 관측(로컬) 좌표계로 변환하려고 합니다.
역변환은 변환 행렬의 역을 통해 얻을 수 있으므로,
세계 공간에서 관측 공간으로의 좌표 변환 행렬은 W^{-1} 입니다

$$W = RT = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ Q_x & Q_y & Q_z & 1 \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ Q_x & Q_y & Q_z & 1 \end{bmatrix}$$

$$w = \frac{T - Q}{\|T - Q\|} \quad u = \frac{j \times w}{\|j \times w\|}$$

$$v = w \times u$$

$$V = W^{-1} = (RT)^{-1} = T^{-1}R^{-1} = T^{-1}R^T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -Q_x & -Q_y & -Q_z & 1 \end{bmatrix} \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ -Q_x \cdot u & -Q_y \cdot v & -Q_z \cdot w & 1 \end{bmatrix}$$

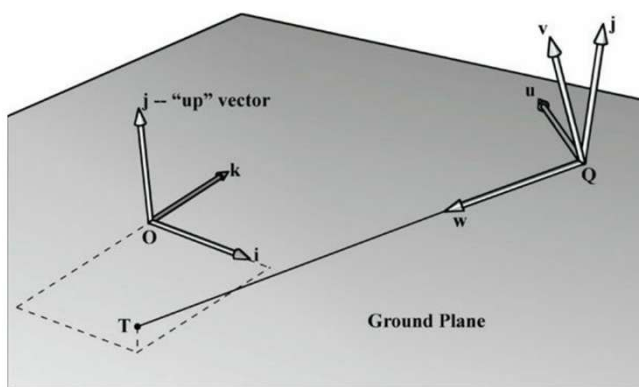
w 와 u는 서로 직교하는 단위 벡터이므로,
v 도 단위 벡터여야 합니다.
따라서 벡터 v를 정규화할 필요가 없습니다

DirectXMath View Transform

$$\mathbf{w} = \frac{\mathbf{T} - \mathbf{Q}}{\|\mathbf{T} - \mathbf{Q}\|} \quad \mathbf{u} = \frac{\mathbf{j} \times \mathbf{w}}{\|\mathbf{j} \times \mathbf{w}\|}$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

w 와 u는 서로 직교하는 단위 벡터이므로, v 도 단위 벡터여야 합니다.
따라서 벡터 v를 정규화할 필요가 없습니다



DirectXMath 라이브러리는 지금까지 설명한 View 행렬을 계산하는 과정에 대해 다음과 같은 함수를 제공합니다.

```
inline XMATRIX XM_CALLCONV XMMatrixLookAtLH // 출력 뷰 변환행렬V
(
    FXMVECTOR EyePosition,           // 카메라 좌표
    FXMVECTOR FocusPosition,        // 카메라 At 좌표
    FXMVECTOR UpDirection           // 카메라 Up좌표
)
```

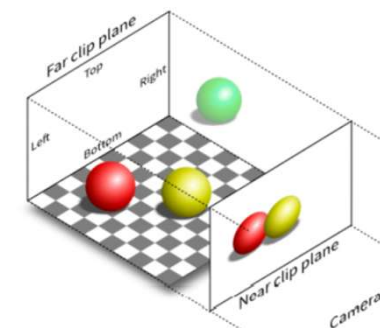
DirectXMath Orthographic View

원점을 중심으로 하는 정사영 행렬을 구해 보겠습니다.
뷰잉 큐브의 너비를 w, 높이를 h, 근거리 평면을 z=n, 원거리 평면을 z=f로 정의합니다

$$\begin{bmatrix} \frac{w}{2} & \frac{h}{2} & n & 1 \\ 0 & 0 & f & 1 \end{bmatrix} P_{ortho} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

x와 y는 크기 조정에만 영향을 받고 w는 절대 변경되지 않으므로
정사 투영 행렬의 매개변수를 다음과 같이 결정할 수 있습니다.
NDC 좌표계이니깐 그렇습니다.

$$P_{ortho} = \begin{bmatrix} \frac{2}{w} & 0 & ? & 0 \\ 0 & \frac{2}{h} & ? & 0 \\ 0 & 0 & ? & 0 \\ 0 & 0 & ? & 1 \end{bmatrix}$$



Orthographic projection (0)

이제 다음 두 가지 공식을 고려해 보세요

$$\begin{bmatrix} \frac{w}{2} & \frac{h}{2} & n & 1 \\ 0 & 0 & f & 1 \end{bmatrix} \begin{bmatrix} ? \\ ? \\ ? \\ ? \end{bmatrix} = 0$$

$$\begin{bmatrix} 0 & 0 & f & 1 \\ 0 & 0 & f & 1 \end{bmatrix} \begin{bmatrix} ? \\ ? \\ ? \\ ? \end{bmatrix} = 1$$

좌표점에 세 번째 행을 곱한 결과는 벡터의 z 값이며,
위 방정식으로 구한 z 값은 모두 상수이고, w와 h와는 명백히 독립적입니다.
따라서 첫 번째 행과 두 번째 행은 모두 0이고,
세 번째 행과 네 번째 행을 각각 A와 B로 설정합니다

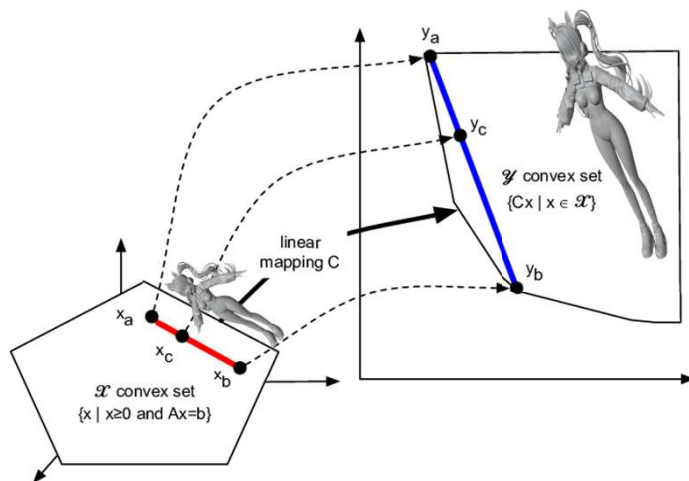
$$P_{ortho} = \begin{bmatrix} \frac{2}{w} & 0 & 0 & 0 \\ 0 & \frac{2}{h} & 0 & 0 \\ 0 & 0 & A & 0 \\ 0 & 0 & B & 1 \end{bmatrix}$$

DirectXMath Orthographic Transform

- Linear mapping (선형 사상)

- 벡터 공간과 벡터 공간 사이를 이어주는 규칙
- A,B를 선형사상 시키는 **이 식**을 이해하기 위해 -1 ~ 1 사이의 선형 사상을 이해해야합니다

$$\begin{array}{l} nA + B = 0 \\ fA + B = 1 \end{array} \quad \begin{array}{l} -1 \leq x \leq 1 \\ -1 \leq y \leq 1 \\ 0 \leq z \leq 1 \\ w = 1 \end{array} \quad \begin{array}{l} x=a \rightarrow x'=-1 \\ x=b \rightarrow x'=+1 \end{array}$$



1. 선형식 가정

$$x' = \alpha x + \beta$$

2. 선형식 가정

$$x = a \mapsto x' = -1 \quad x = b \mapsto x' = +1$$

$$-1 = \alpha a + \beta$$

$$+1 = \alpha b + \beta$$

3. 두 식을 풀기

$$(1) - (-1) = (\alpha b + \beta) - (\alpha a + \beta) \quad -1 = \frac{2}{b-a} \cdot a + \beta$$

$$2 = \alpha(b-a)$$

$$\beta = -1 - \frac{2a}{b-a}$$

$$\alpha = \frac{2}{b-a}$$

4. 최종 식

$$x' = \frac{2}{b-a} \left(x - \frac{a+b}{2} \right)$$

DirectXMath Orthographic View

A, B를 n과 f로 나타내보겠습니다

$$\begin{aligned} nA + B &= 0 \\ fA + B &= 1 \end{aligned} \longrightarrow \begin{aligned} A &= \frac{1}{f-n} \\ B &= \frac{-n}{f-n} \end{aligned} \longrightarrow P_{ortho} = \begin{bmatrix} \frac{2}{w} & 0 & 0 & 0 \\ 0 & \frac{2}{h} & 0 & 0 \\ 0 & 0 & \frac{1}{f-n} & 0 \\ 0 & 0 & \frac{-n}{f-n} & 1 \end{bmatrix} \quad G(z) = \frac{z}{f-n} + \frac{-n}{f-n}$$

DirectXMath에서 해당 함수는 다음과 같습니다

XMMatrixOrthographicLH

```
inline XMATRIX XM_CALLCONV XMMatrixOrthographicLH
(
    float ViewWidth,           // [In] 투영 영역의 너비
    float ViewHeight,          // [In] 투영 영역의 높이
    float NearZ,                // [In] 근거리 평면
    float FarZ                  // [In] 원거리 평면
)
```

투영 큐브가

[왼쪽, 오른쪽] x [아래쪽, 위쪽] x [n, f]로 지정되면 다음과 같습니다

$$P_{ortho} = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & 0 \\ 0 & \frac{2}{top-bottom} & 0 & 0 \\ 0 & 0 & \frac{1}{f-n} & 0 \\ -\frac{right+left}{right-left} & -\frac{top+bottom}{top-bottom} & \frac{-n}{f-n} & 1 \end{bmatrix}$$

DirectXMath에서 해당 함수는 다음과 같습니다

XMMatrixOrthographicOffCenterLH

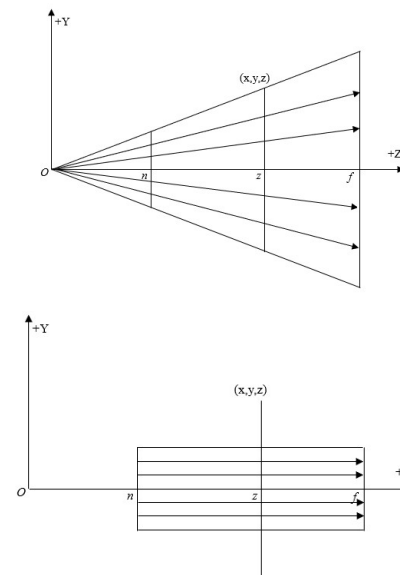
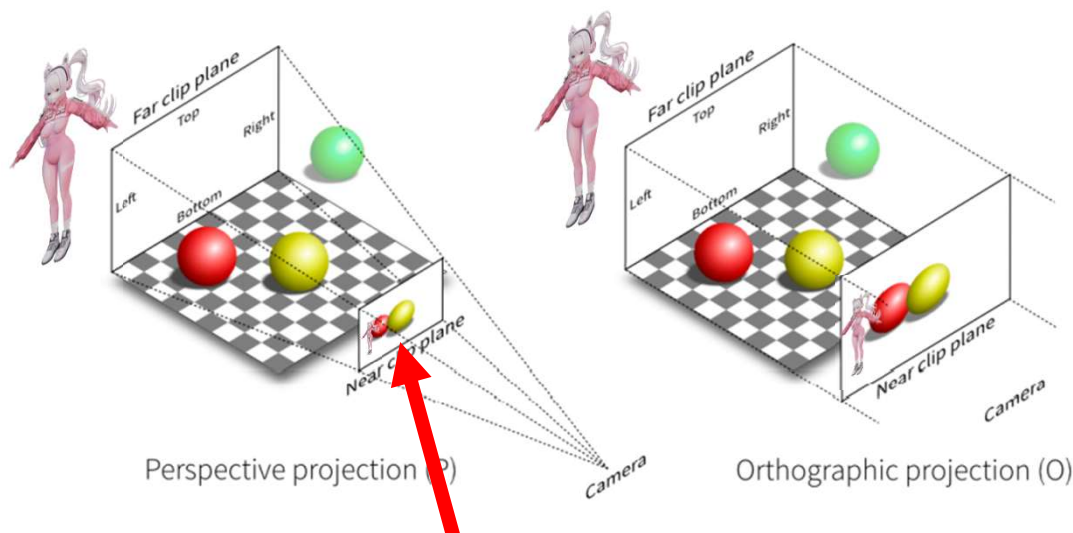
```
inline XMATRIX XM_CALLCONV XMMatrixOrthographicOffCenterLH
(
    float ViewLeft,           // [In] 투영 영역의 왼쪽 가장자리
    float ViewRight,          // [In] 투영 영역의 오른쪽 가장자리
    float ViewBottom,         // [In] 투영 영역의 아래쪽 가장자리
    float ViewTop,            // [In] 투영 영역의 위쪽 가장자리
    float NearZ,               // [In] 근거리 평면
    float FarZ                 // [In] 원거리 평면
)
```

DirectXMath Projection Transform

투영 변환의 목적은 3D 장면을 2D 평면에 투영하는 것입니다.

투영에는 원근 투영 과 정사 투영 의 두 가지 유형이 있습니다.

원근 투영에서는 물체가 가까이 있을 때는 더 크게 보이고, 멀리 있을 때는 더 작게 보입니다



NDC(Normalized Device Coordinate) : 플레이어가 보게 되는 최종 2D 이미지

$$\begin{aligned} -1 &\leq x \leq 1 \\ -1 &\leq y \leq 1 \\ 0 &\leq z \leq 1 \\ w &= 1 \end{aligned}$$

이 범위는 정육면체를 나타내기도 합니다.

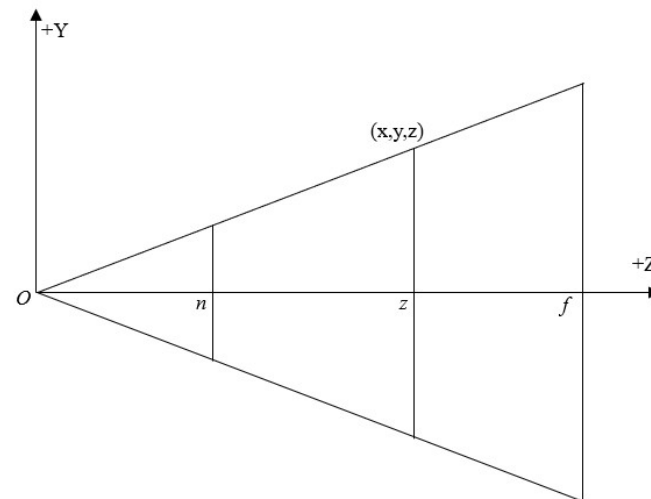
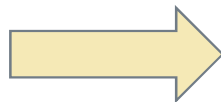
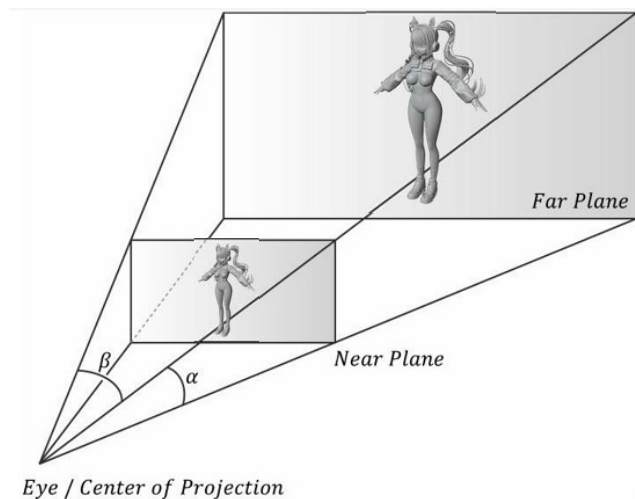
이 범위를 사용하면 정육면체를 원하는 크기와 위치의 2D 직사각형으로 변환할 수 있습니다.

이 변환 과정을 뷰포트 변환 이라고 합니다

DirectXMath Perspective View

정사투영과 마찬가지로, 거리를 제한하기 위해 근거리 평면과 원거리 평면도 정의해야 합니다. 정의된 가시 범위는 실제로 절두체(혹은 관측원뿔이라고 하는 것이 더 편리할 수 있음)입니다. 하지만 이 뷰잉 콘은 다루기가 쉽지 않습니다.

이 뷰잉 콘을 직교하는 정육면체에 압축하여 직교 투영 변환을 수행하는 것이 가장 좋습니다 .



DirectXMath Perspective View

같은 높이의 물체는 가까이 있을 때는 더 크게 보이고 멀리 있을 때는 더 작게 보입니다.
 님삼각형의 성질을 바탕으로 (x, y, z) 가 평면 $z = n$ 에 투영될 때 위치 (x', y', n) 을 구할 수 있습니다

$$\frac{x'}{n} = \frac{x}{z}, \quad \frac{y'}{n} = \frac{y}{z}$$

$$x' = \frac{xn}{z}, \quad y' = \frac{yn}{z}$$

행렬 곱은 선형이라 $1/z$ 같은 나눗셈을 직접 만들 수 없습니다.

그래서 동차 좌표(homogeneous coordinate)를 씁니다

동차좌표 한 번에 표현하려고 “원근 나눗셈 전” 벡터를 이렇게 상상할 수 있습니다

$$[X', Y', Z', W'] \xrightarrow{\text{분할}} \left(\frac{X'}{W'}, \frac{Y'}{W'}, \frac{Z'}{W'}, 1 \right) = \left(\frac{xn}{z}, \frac{yn}{z}, \dots, 1 \right).$$

$W'=z$ 이 되게 하면, $X'=nx$, $Y'=ny$, $Z'=nz$ 로 잡는 것만으로

$X'/W'=xn/z$, $Y'/W'=yn/z$, $Z'/W'=zn/z=xn/z$, $Y'/W'=yn/z$ 가 자동으로 됩니다

이제 “persp→ortho(시야절두체를 직교 박스로 ‘퍼는’ 단계)”에서는 x, y 만 눌러 주고 z 는 그대로 두면 됩니다

즉, 분할 후 z 가 그대로 z 가 되게 하려면

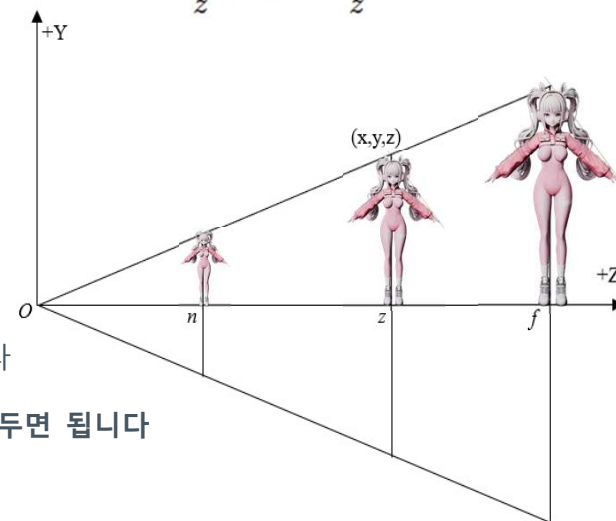
$$\frac{Z'}{W'} = z \Rightarrow Z' = z \cdot W' = z \cdot z = z^2.$$

그런데 일반 z 에 대해 z^2 을 선형 행렬로는 직접 만들 수 없죠(비선형이니까).

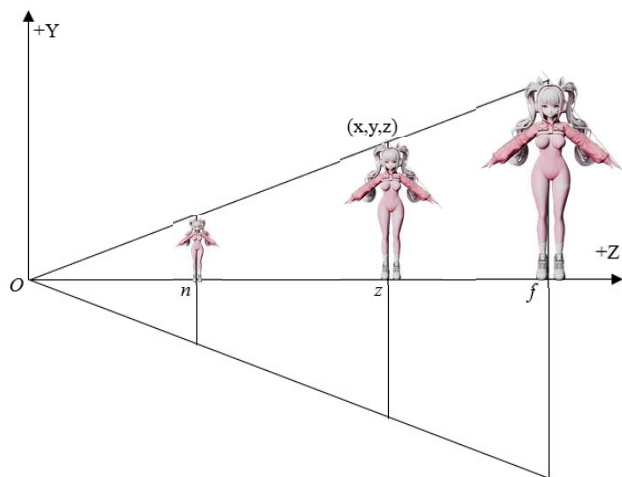
그래서 특정 두 단면에서만 조건을 걸어 선형식을 맞춥니다

참고 : 동차좌표에서는 (X, Y, Z, W) 와 (kX, kY, kZ, kW) 가 같은 점을 나타냅니다.

-> 동차좌표의 “배수는 같음” 성질



DirectXMath Perspective View



$z = n$ 이라고 하고, 근평면 $(x, y, n, 1)$ 위의 한 점을 잡습니다.
그러면 (xn, yn, n^2, n) 도 이 점을 나타내며,
근평면 위의 점은 여전히 근평면으로 투영됩니다(돌출은 발생하지 않습니다).
따라서 다음과 같은 식이 성립합니다

$$\begin{bmatrix} x & y & n & 1 \end{bmatrix} P_{\text{persp} \rightarrow \text{ortho}} = \begin{bmatrix} xn & yn & n^2 & n \end{bmatrix}$$

$$\begin{bmatrix} x & y & f & 1 \end{bmatrix} P_{\text{persp} \rightarrow \text{ortho}} = \begin{bmatrix} xn & yn & f^2 & f \end{bmatrix}$$

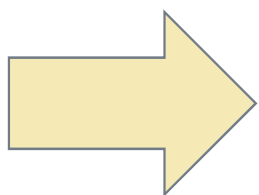
x 와 y 모두 스케일링되므로, 해당 피벗만 n 입니다.
 w 요소의 경우, 이후에 설명 합니다.
아래에서 네 번째 열의 세 번째 행을 n 대신 1로 설정합니다.

$$P_{\text{persp} \rightarrow \text{ortho}} = \begin{bmatrix} n & 0 & ? & 0 \\ 0 & n & ? & 0 \\ 0 & 0 & ? & 1 \\ 0 & 0 & ? & 0 \end{bmatrix}$$

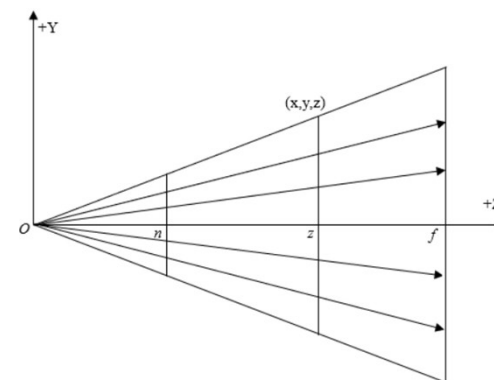
DirectXMath Perspective View

이 두 방정식을 결합하면, 여기서는 네 번째 열의 세 번째 행만 1이 될 수 있다는 것을 알 수 있습니다.
이제 z 구성요소의 변환을 관찰하세요

$$\begin{aligned} \begin{bmatrix} x & y & n & 1 \end{bmatrix} \begin{bmatrix} ? \\ ? \\ ? \\ ? \end{bmatrix} &= n^2 \\ \begin{bmatrix} x & y & f & 1 \end{bmatrix} \begin{bmatrix} ? \\ ? \\ ? \\ ? \end{bmatrix} &= f^2 \end{aligned} \longrightarrow P_{\text{persp} \rightarrow \text{ortho}} = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & A & 1 \\ 0 & 0 & B & 0 \end{bmatrix} \longrightarrow \begin{aligned} nA + B &= n^2 \\ fA + B &= f^2 \end{aligned} \longrightarrow \begin{aligned} A &= n + f \\ B &= -fn \end{aligned}$$



$$P_{\text{persp} \rightarrow \text{ortho}} = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n + f & 1 \\ 0 & 0 & -fn & 0 \end{bmatrix} \quad G(z) = A + \frac{B}{z}$$

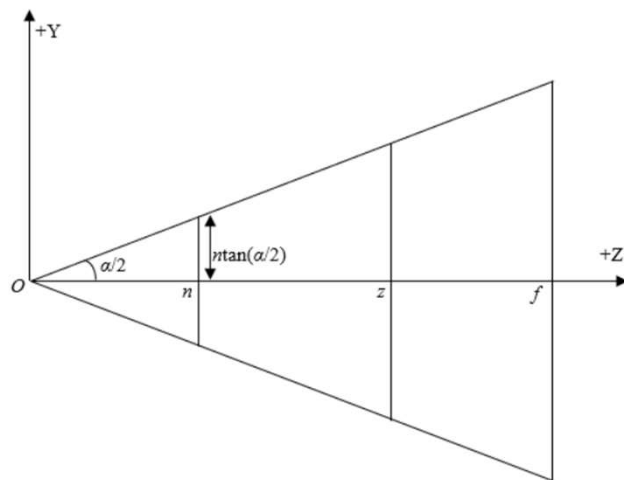


DirectXMath FOV (Field of View)

[l, r]x[b, t]x[n, f]로 투영 행렬을 정의할 수 있지만 직관적이지는 않습니다.
 이를 더 간단히 표현하기 위해 두 개의 새로운 변수를 도입합니다.
 수직 시야(FOV)는 카메라가 위아래로 볼 수 있는 최대 각도입니다.
 이는 근거리 평면의 높이를 간접적으로 정의합니다.

시야 절두체의 근평면의 너비
 (직교 육면체의 너비와 높이이기도 함)도
종횡비에 의해 결정됩니다.
 종횡비를 r 이라고 하면 다음과 같습니다

$$h = 2n \cdot \tan(\alpha/2) \quad r = w/h \quad w = 2rn \cdot \tan(\alpha/2)$$



$$P_{\text{ortho}} = \begin{bmatrix} \frac{1}{rn \cdot \tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{n \cdot \tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & \frac{1}{f-n} & 0 \\ 0 & 0 & \frac{-n}{f-n} & 1 \end{bmatrix}$$

$$P_{\text{persp}} = P_{\text{persp} \rightarrow \text{ortho}} P_{\text{ortho}}$$

$$= \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & 1 \\ 0 & 0 & -fn & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{rn \cdot \tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{n \cdot \tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & \frac{1}{f-n} & 0 \\ 0 & 0 & \frac{-n}{f-n} & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \frac{1}{r \tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & \frac{f}{f-n} & 1 \\ 0 & 0 & -\frac{nf}{f-n} & 0 \end{bmatrix}$$

DirectXMath FOV (Field of View)

투영 작업이 완료되면,
 원근 투영 변환을 거친 모든 점들이
 동차 클리핑 공간(Homogeneous Clip Space)[x, y, z, w] 으로 변환됩니다
 w는 실제로 관찰 공간의 깊이 값이기 때문에 w는 0보다 작거나 같아서는 안 됩니다

$$\begin{aligned} -w &\leq x \leq w \\ -w &\leq y \leq w \\ 0 &\leq z \leq w \\ 0 &< w \end{aligned}$$

로컬 공간 → 월드 공간 → 뷰 공간 → 균질 클리핑 공간 → 정규화된 디바이스 공간(NDC)
 각각: 세계 행렬 → 관찰(뷰) 행렬 → 원근 행렬 → 원근 분할

원근/직교 차이

원근 투영: $w \neq 1 \rightarrow$ 균질 클리핑 공간 $\neq$ NDC

직교 투영: $w = 1 \rightarrow$ 균질 클리핑 공간 = NDC

깊이 값 정규화

$z \in [n, f] \rightarrow g(z) \in [0, 1]$ (순서 보존)

정규화 후에도 깊이 관계는 유지됨 → 깊이 버퍼 비교 가능

단순 스케일+이동으로는 처리 불가 → 투영 행렬 내부에 통합됨

DirectXMath에서는 다음 함수를 사용하여 투영 행렬을 구합니다

```
// 투영 행렬을 반환합니다.
inline XMATRIX XM_CALLCONV XMMatrixPerspectiveFovLH (
    float FovAngleY,           // 중앙 수직 각도
    float AspectRatio,         // 종횡비
    float NearZ,                // 근거리 평면 거리
    float FarZ                  // 원거리 평면 거리
)
```

상수 버퍼 (Constant Buffer)

› HLSL에서 상수 버퍼 변수는 C++의 전역 상수와 유사하며 셰이더 코드에서 사용됩니다

HLSL 상수 버퍼의 예시

```
cbuffer ConstantBuffer : register(b0)
{
    matrix g_World;
    matrix g_View;
    matrix g_Proj;
}
```

C++ 상수버퍼 구조체 예시

```
struct ConstantBuffer
{
    DirectX::XMMATRIX world;
    DirectX::XMMATRIX view;
    DirectX::XMMATRIX proj;
};
```

상수 버퍼 (Constant Buffer)

› 런타임에 상수 버퍼를 업데이트하는 방법은 두 가지가 있습니다

- 1. 리소스를 생성할 때 Usage이것을 지정 D3D11_USAGE_DEFAULT하면 GPU에서 상수 버퍼를 쓸 수 있으며 ID3D11DeviceContext::UpdateSubresource메서드를 사용하여 업데이트해야 합니다.
- 2. 리소스를 생성할 때, CPU에서 상수 버퍼를 쓸 수 있도록 Usage, D3D11_USAGE_DYNAMIC를 지정합니다. 먼저 메서드를 통해 메모리 매핑을 가져온 다음, 매핑된 메모리 영역에 업데이트하고, 마지막으로 메서드를 통해 점유를 해제합니다.

CPUAccessFlagsD3D11_CPU_ACCESS_WRITEID3D11DeviceContext::MapID3D11DeviceContext::Unmap

```
D3D11_BUFFER_DESC cbd;  
ZeroMemory(&cbd, sizeof(cbd));  
cbd.Usage = D3D11_USAGE_DYNAMIC;  
cbd.ByteWidth = sizeof(ConstantBuffer);  
cbd.BindFlags = D3D11_BIND_CONSTANT_BUFFER;  
cbd.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;  
// 초기 데이터를 사용하지 않고 새 상수 버퍼를 생성합니다.  
HR(m_pd3dDevice->CreateBuffer(&cbd, nullptr,  
m_pConstantBuffer.GetAddressOf()));
```

참고 :

상수 버퍼를 생성할 때 설명 매개변수는

ByteWidth 16의 배수여야 합니다.

HLSL 상수 버퍼 자체와 읽기 및 쓰기 작업은
16바이트에 엄격하게 맞춰야 하기 때문입니다.

- 버퍼에 있는 데이터에 대한 포인터를 가져오고 GPU가 버퍼에 액세스하는 것을 거부합니다.

› ID3D11DeviceContext::Unmap 함수

- 리소스에 대한 포인터를 무효화하고 리소스에 대한 GPU 액세스를 다시 활성화합니다.
- 버퍼에 있는 데이터에 대한 포인터를 가져오고 GPU가 버퍼에 액세스하는 것을 거부합니다.

```
void ID3D11DeviceContext::Unmap(
    ID3D11Resource* pResource,          // [In] ID3D11Resource 인터페이스를 포함하는 리소스 객체
    UINT Subresource                     // [In] 서브리소스 인덱스, 버퍼 리소스는 0
);
```

상수 버퍼 - 다이내믹 업데이트

- › **mBuffer**구조체 변수를 사용하여 상수 버퍼를 업데이트해야 합니다.
 - HLSL 열 주 행렬의 전치를 상쇄하기 위해 합 view행렬을 proj미리 한 번 전치해야 합니다.
 - world해당 행렬이 이미 단위 행렬이므로 이 작업은 필요하지 않습니다

```
m_CBuffer.world = XMMatrixIdentity();// 단위 매트릭스의 조기는 그 자체입니다
```

```
m_CBuffer.view = XMMatrixTranspose(XMMatrixLookAtLH(  
    XMVectorSet(0.0f, 0.0f, -5.0f, 0.0f),  
    XMVectorSet(0.0f + 0.0f, 0.0f + 0.0f, -5.0f + 1.0f, 0.0f),  
    XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f)  
));  
m_CBuffer.proj = XMMatrixTranspose(XMMatrixPerspectiveFovLH(XM_PIDIV2, AspectRatio(), 1.0f, 1000.0f));
```

Rasterizer Stage – Viewport 매핑

- › NDC 좌표 → 화면 공간으로 매핑
- › ViewportInfo 설정만으로 동작 (고정 기능)
- › Viewport 밖의 좌표는 프래그먼트 생성 안 함

```
/*  
 * @brief 뷰포트(Viewport) 설정  
 * @details  
 *   - TopLeftX/Y : 출력 영역의 시작 좌표 (0,0 → 좌상단)  
 *   - Width/Height : 윈도우 클라이언트 크기 기준  
 *   - MinDepth/MaxDepth : 깊이 범위 (보통 0.0 ~ 1.0)  
 *   - RSetViewports : 래스터라이저 스테이지에 뷰포트 바인딩  
 */  
D3D11_VIEWPORT viewport = {};  
viewport.TopLeftX = 0;  
viewport.TopLeftY = 0;  
viewport.Width = (float)m_ClientWidth;  
viewport.Height = (float)m_ClientHeight;  
viewport.MinDepth = 0.0f;  
viewport.MaxDepth = 1.0f;  
m_pDeviceContext->RSetViewports(1, &viewport);
```

참고

<https://nikke-kr.com/mmd.html>

[Vulkan Tutorial]

https://vulkan-tutorial.com/Vertex_buffers/Vertex_input_description

[Input-Assembler]

<http://learn.microsoft.com/ko-kr/windows/win32/direct3d11/d3d10-graphics-programming-guide-input-assembler-stage>

[vertex shader]

<https://docs.tizen.org/application/native/guides/graphics/vertex-shader/>

<https://learn.microsoft.com/ko-kr/windows/win32/direct3d11/vertex-shader-stage>

[pixel shader]

<https://learn.microsoft.com/ko-kr/windows/win32/direct3d11/pixel-shader-stage>

<https://docs.tizen.org/application/native/guides/graphics/fragment-shader/>

[Orthographic projection]

https://en.wikipedia.org/wiki/Orthographic_projection

[Transform]

<https://directx11.tech/#/part1/04?id=%e9%bd%90%e6%ac%a1%e8%a3%81%e5%89%aa%e7%a9%ba%e9%97%b4%e3%80%81%e5%bd%92%e4%b8%80%e5%8c%96%e6%b7%b1%e5%ba%a6>

